

Writing Sample

Luca Prigione

MSc Computer Science - Foundations of Computing and Concurrency

1 Introduction

When analyzing computational problems, one of the main aspects is their complexity and, consequently, the possibility of solving them in an efficient way. In this context there are multiple notable problems that are precisely studied because of their computational complexity and for which no efficient algorithms are known, belonging to the complexity class NP. One of the most studied problems is the Hitting Set problem [5].

The Hitting Set problem consists in, given a set of elements and a collection of subsets, determining a minimal subset of elements such that it intersects with all the given subsets, in other words, such that it contains at least one element belonging to each of them. This problem is known to be NP-hard [4] and is closely connected to numerous combinatorial optimization problems. A natural representation of the problem is via the use of hypergraphs, data structures in which each hyperedge may connect to an arbitrary number of nodes, allowing it to model more complex relationships than traditional graphs [1].

This work considers a variant of the Hitting Set problem characterized by a temporal dimension and by an online context, within which the input evolves over time [3]. The objective is to maintain a valid solution to the problem even as the problem changes dynamically, while trying to avoid re-computing the entire solution on every update. This introduces new challenges in terms of both computational efficiency and quality and stability of the solutions over time.

The objective of this work is therefore to study and evaluate various algorithmic approaches to the problem, analyzing both their theoretical properties and their behaviour in practice. Particular attention is given to the relationship between solution quality, update costs and execution time, so as to understand the trade-offs in dynamic contexts and to find, when possible, the optimal solution.

2 Problem Definition

In this work we consider a variant of the Hitting Set problem in a dynamic and temporal context. The input is modeled as a temporal hypergraph $H = (V, E)$, where V represents the set of vertices or nodes and E is a collection of hyperedges. Each hyperedge is a tuple (S, t) , where $S \subseteq V$ is a subset of vertices and t represent a discrete instant in time.

Given a time interval of length ε , we define a sliding time window such that, at any instant t , the set of active hyperedges is given by:

$$W(t) = \{(S, t') \in E \mid t - \varepsilon \leq t' \leq t\}. \quad (1)$$

The objective is to keep, for every instant in time t , a set of vertices $X(t) \subseteq V$ that constitutes a hitting set for all active hyperedges, meaning it is such that:

$$\forall (S, t') \in W(t), \quad X(t) \cap S \neq \emptyset. \quad (2)$$

Contrary to the static version of the problem, in this context the solution must be updated over time, accounting for the addition of new hyperedges and the removal of those that become no longer active.

The objective is then to keep such a valid solution while optimizing multiple criteria at the same time:

- Solution size: minimizing $|X(t)|$;
- Update cost:

$$\Delta X(t) = |X(t) \setminus X(t-1)| + |X(t-1) \setminus X(t)|. \quad (3)$$

- Computational efficiency: minimizing the time needed to update the solution.

In summary, the problem consists in keeping a hitting set that is small, stable and efficient in a dynamic and online context.

3 Algorithms and Approaches

To tackle the above problem, multiple different algorithmic approaches were developed and analyzed, and their behaviour in terms of solution quality, stability and efficiency compared.

An already existing algorithm based on the PySAT library, in particular on the module Hitman, was used as a reference [2]. This approach served as a baseline to compare with the performance of the developed algorithms.

In addition to this, six heuristic algorithms were implemented, divided into two main categories.

The first category consists of variants of the classic greedy approach based on the degree of the nodes in the graph. These algorithms iteratively select the node that has the highest degree or otherwise covers the largest amount of hyperedges that are not yet covered. Three variants of this approach were developed, which differ in the level of optimization and in the way the data structure is updated during execution.

The second category includes algorithms based on a local strategy. The hyperedges are iterated through and, upon finding an uncovered hyperedge, a node belonging to it is selected, typically the one with the most connections. In this case, three variants were also developed, with different ways of handling the local information and updating the solution.

This approach is able to explore multiple strategies to construct the solution, comparing global methods that enjoy the privilege of a complete view of the problem, with local methods, which operate on individual constraints. It is therefore possible to better analyze the trade-offs between solution quality, computational cost and stability in the dynamic system.

4 Experimental Evaluation

To evaluate the proposed algorithms, their performance was tested on large instances of the problem, containing over one million nodes and more than twenty-thousand hyperedges. This allows the behaviour of the algorithms can be analyzed in realistic and computationally significant scenarios.

The PySAT-based approach, capable of calculating a hitting set that is optimal or close to optimal, was used as a reference. In the aforementioned instance of the problem, this method produced a solution of size 17676 in about 24 seconds.

The greedy algorithms based on the global degree of the nodes showed particularly interesting results: they produced solutions of 17681 nodes, extremely close to optimal. However this level of quality came with a significantly higher computational cost. To be precise, the un-optimized version took over 300 seconds, while the optimized versions reduced the time to about 63 seconds, with the quality of their solutions staying the same.

The approaches based on a local strategy showed different behaviour.

These algorithms produced slightly worse solutions in terms of size, between 17683 and 17695 nodes, but were extremely efficient from a computational point of view, requiring time in the order of milliseconds to construct their solutions.

These results clearly showcase a trade-off between solution quality and computational complexity. The global approaches provide solutions that are very close to optimal, with the downside of higher execution times, while the local approaches partially sacrificed optimality to obtain superior performance in terms of time.

A relevant aspect of these results emerges when considering the dynamic context introduced by the sliding time window model. In such a scenario the input evolves constantly and, at every advancement of the time window, the set of active hyperedges changes.

If an approach based on completely re-computing the solution at every update was adopted, even an optimal algorithm would need to be executed repeatedly for every shift of the time window. This would make for a very high overall computational cost, growing linearly with the amount of updates, making such solutions difficult to apply in dynamic or real-time contexts.

Meanwhile, the proposed heuristic approaches are capable of using the structure of the previous solution to update the hitting set incrementally, without needing to re-compute it from zero at every change. This provides a significant improvement in terms of overall computational efficiency, while still maintaining an acceptable solution quality.

This further highlights the importance of the trade-offs between optimality and efficiency in the online context that is considered, showing how non-optimal solutions can be preferable when accounting for the dynamics of the problem.

5 Conclusion

This work studies a variant of the Hitting Set problem in a dynamic and temporal context, with emphasis on online scenarios where the input evolves over time. Multiple different algorithmic approaches were developed and analyzed, for the purpose of understanding the behaviour of various strategies in terms of solution quality, stability and efficiency.

The experimental results show how greedy global approaches are effective for finding results that are close to optimal, while local strategies are especially apt for contexts where fast and continuous results are required,

such as in the case of dynamic systems.

Overall, the work shows how it is possible to balance the various objectives effectively, highlighting the importance of the trade-off between quality of solutions and computational cost. These two aspects are particularly relevant in the context of dynamic and online problems, where the ability to rapidly update a solution can be more important than reaching absolute optimality.

References

- [1] Claude Berge. *Hypergraphs: Combinatorics of Finite Sets*. 1984.
- [2] Armin Biere et al. The sat solver pysat. 2019.
- [3] Allan Borodin and Ran El-Yaniv. *Online Computation and Competitive Analysis*. 1998.
- [4] Richard M. Karp. Reducibility among combinatorial problems. *Complexity of Computer Computations*, 1972.
- [5] Vijay V. Vazirani. *Approximation Algorithms*. 2001.